Final RTOS-Snake Report

Team Members:

- Alireza Fathihafshejani: afathihafs@stud.hs-heilbronn.de
- Keerthi Meghana Alaparthi: kalaparthi@stud.hs-heilbronn.de
- Nithish Kodavatikanti: nkodavatik@stud.hs-heilbronn.de

Split Works:

- Measuring, setting architecture, Coding Zephyr, Chapter 4-6 report writing: Alireza Fathihafshejani
- Coding FreeRTOS, chapter 1-3 report writing: Nithish Kodavatikanti
- Coding ThreadX, chapter 1-3 report writing: Keerthi Meghana Alaparthi

Declaration on the Use of Generative AI

Our team made limited use of generative artificial intelligence (AI) tools during the preparation of this report. OpenAI ChatGPT (GPT-5.5, accessed between May 2026 and June 2026) was used exclusively to assist with language refinement, grammar correction, text restructuring, formatting suggestions, figures crafting and brainstorming activities related to the presentation of the research.

The AI system was not used to generate experimental data, perform measurements, conduct analyses, implement software, interpret results, or formulate the scientific contributions of this work. All technical content, experimental procedures, benchmark implementations, data collection, data evaluation, and conclusions were developed, verified, and approved by the author.

Chapter 1 – Introduction

1.1 Introduction

Embedded systems have become an essential component of modern electronic devices and industrial applications. They are widely used in automotive systems, industrial automation, medical equipment, robotics, aerospace platforms, and Internet of Things (IoT) devices. Many of these applications must react to external events within strict timing constraints, making predictable and deterministic behaviour a critical system requirement.

To satisfy these timing requirements, Real-Time Operating Systems (RTOSs) are commonly employed. Unlike general-purpose operating systems, RTOSs are designed to provide deterministic task scheduling, low interrupt latency, and predictable response

times. These characteristics enable embedded systems to perform time-critical operations while maintaining system reliability and responsiveness.

Several RTOS solutions are currently available for embedded applications. Among the most widely adopted are FreeRTOS, Zephyr RTOS, and Azure RTOS ThreadX. Each operating system offers different architectural approaches, scheduling mechanisms, and design objectives. FreeRTOS is known for its lightweight design and widespread industrial adoption, Zephyr provides a highly modular architecture suitable for modern IoT applications, while ThreadX emphasizes deterministic performance and low scheduling overhead for real-time environments.

As embedded systems continue to evolve, distributed architectures consisting of multiple interconnected processing nodes are becoming increasingly common. In such systems, communication latency between nodes can significantly influence overall system performance. While many previous studies have evaluated RTOS performance on individual microcontrollers, relatively limited research has investigated latency behaviour in distributed communication chains involving multiple RTOS platforms operating together.

This research addresses that challenge by implementing a distributed benchmark architecture using STM32 Nucleo-F439ZI development boards running FreeRTOS, Zephyr, and Azure RTOS ThreadX. The benchmark measures end-to-end communication latency as a signal propagates through different RTOS forwarding nodes and returns to a dedicated measurement node. Hardware cycle counters are used to obtain precise latency measurements, enabling a direct comparison of timing performance across different communication paths.

The results of this study provide insight into the latency characteristics, timing variability, and determinism of heterogeneous RTOS environments operating within a distributed embedded architecture. The findings may assist embedded system designers in selecting appropriate RTOS solutions for applications requiring predictable communication performance and low-latency operation.

Chapter 2 –Literature Review

2.1 Introduction

Real-Time Operating Systems (RTOSs) have become a fundamental component of modern embedded systems. Their ability to provide deterministic scheduling, low interrupt latency, and predictable task execution makes them suitable for applications where timing requirements are critical. Examples include industrial automation, automotive control systems, robotics, medical devices, aerospace applications, and Internet of Things (IoT) platforms.

As embedded systems become increasingly distributed, communication latency between processing nodes plays an important role in overall system performance.

Consequently, evaluating the timing behaviour of RTOSs has attracted significant attention from researchers and engineers. This chapter reviews the fundamental concepts of RTOSs, discusses the characteristics of FreeRTOS, Zephyr, and Azure RTOS ThreadX, and examines previous studies related to RTOS performance evaluation and latency measurement.

2.2 Real-Time Operating Systems

A Real-Time Operating System is designed to ensure that system operations are executed within predictable timing constraints. Unlike general-purpose operating systems that focus primarily on throughput and user experience, RTOSs prioritize deterministic behaviour and timely task completion.

Typical RTOS services include:

- Task scheduling
- Interrupt management
- Inter-task communication
- Synchronization mechanisms
- Memory management
- Timer services

The effectiveness of an RTOS is often evaluated through metrics such as interrupt latency, context-switching overhead, task response time, and timing determinism. These metrics are particularly important in applications where delayed responses may lead to performance degradation or system failure.

2.2.1 FreeRTOS

FreeRTOS is one of the most widely used open-source RTOSs in embedded applications. It was designed to provide a lightweight and efficient kernel suitable for resource-constrained microcontrollers.

The operating system employs a priority-based pre-emptive scheduler and provides mechanisms such as queues, semaphores, mutexes, event groups, and task notifications. Due to its small memory footprint and simplicity, FreeRTOS is widely adopted in industrial control systems, consumer electronics, and IoT devices.

Several studies have demonstrated that FreeRTOS offers low scheduling overhead and predictable timing behaviour while maintaining a relatively simple software architecture. These characteristics make it suitable for real-time applications requiring deterministic operation on low-cost hardware platforms.

2.2.2 Zephyr RTOS

Zephyr is an open-source RTOS maintained by the Linux Foundation. It was developed to address the requirements of modern embedded and IoT systems while providing scalability across a wide range of hardware platforms.

Unlike traditional lightweight RTOS implementations, Zephyr includes extensive networking support, hardware abstraction layers, device drivers, security features, and configurable kernel services. Zephyr supports both cooperative and pre-emptive scheduling models and utilizes a Device Tree-based hardware configuration framework.

Although the additional abstraction layers can introduce some overhead compared to lightweight kernels, Zephyr provides significant flexibility and portability. As a result, it has become increasingly popular in connected embedded systems and industrial IoT applications.

2.2.3 Azure RTOS ThreadX

Azure RTOS ThreadX is a commercial-grade real-time operating system originally developed by Express Logic and later acquired by Microsoft. The operating system is designed to provide highly deterministic performance and efficient resource utilization.

ThreadX employs an optimized scheduler and lightweight kernel architecture that minimize context-switching overhead. Advanced scheduling mechanisms such as priority inheritance and pre-emption thresholds contribute to predictable timing behaviour and reduced latency variability.

Because of its deterministic characteristics, ThreadX is frequently used in safety-critical and industrial applications where predictable response times are essential. Previous benchmarking studies have often reported excellent interrupt response performance and low scheduling latency when compared with other RTOS solutions.

2.2.4 Interrupt Latency and Timing Determinism

Interrupt latency represents the delay between the occurrence of an external event and the execution of the corresponding interrupt service routine. In embedded systems, interrupt latency is a critical performance metric because it directly affects system responsiveness.

The total latency associated with an event typically includes:

1. Hardware interrupt detection.
2. Interrupt controller processing.
3. Interrupt service routine execution.
4. Scheduler operation.
5. Context switching.
6. Application-level processing.

Timing determinism refers to the consistency of these response times. A deterministic system produces similar latency values across repeated executions of the same

operation. Therefore, both average latency and latency variation must be considered when evaluating RTOS performance.

2.3 RTOS Benchmarking Techniques

Several approaches have been proposed for evaluating RTOS performance. Common benchmarking techniques include:

- Interrupt latency measurement
- Context-switch latency measurement
- Task-switching benchmarks
- Inter-process communication benchmarks
- End-to-end communication latency measurement

Many benchmark studies rely on software timers to collect timing information. However, hardware cycle counters generally provide higher precision because they measure processor cycles directly without introducing significant software overhead.

The ARM Cortex-M architecture provides the Data Watchpoint and Trace (DWT) cycle counter, which is commonly used for accurate latency measurement. The use of hardware cycle counters enables high-resolution timing analysis and reduces measurement uncertainty.

2.4 Previous Research

Numerous studies have compared the performance of different RTOS platforms running on embedded microcontrollers. Most existing research focuses on individual performance metrics such as interrupt latency, scheduling overhead, memory usage, and context-switching time.

Previous investigations have shown that FreeRTOS provides efficient task scheduling and low resource consumption, making it suitable for resource-constrained systems. Studies involving ThreadX have frequently reported highly deterministic timing behaviour and low interrupt latency due to its optimized kernel architecture.

Research related to Zephyr has demonstrated that the operating system offers excellent scalability and extensive functionality, although its additional abstraction layers may introduce modest performance overhead compared with more lightweight kernels.

Despite the availability of RTOS benchmarking studies, most evaluations are performed on a single microcontroller operating independently. Consequently, relatively few studies examine latency behaviour within distributed communication architectures involving multiple RTOS platforms operating together.

2.5 Research Gap

The review of existing literature reveals that most RTOS performance evaluations focus on isolated systems running a single operating system on a single device. While such studies provide valuable insights into kernel behaviour, they do not fully represent modern distributed embedded environments where multiple processing nodes communicate and cooperate.

Furthermore, limited research has investigated the cumulative latency introduced when signals propagate through heterogeneous communication chains consisting of different RTOS implementations. Understanding this behaviour is important for industrial and IoT applications where timing constraints must be maintained across multiple interconnected devices.

Therefore, a research gap exists regarding the evaluation of end-to-end communication latency in distributed embedded systems involving multiple RTOS platforms. This study addresses that gap by implementing and evaluating a distributed communication architecture based on FreeRTOS, Zephyr, and Azure RTOS ThreadX running on identical STM32 hardware platforms.

2.6 Chapter Summary

This chapter reviewed the fundamental concepts of real-time operating systems and discussed the characteristics of FreeRTOS, Zephyr RTOS, and Azure RTOS ThreadX. Key concepts including interrupt latency, timing determinism, and RTOS benchmarking techniques were examined. Existing studies related to RTOS performance evaluation were also reviewed, highlighting the limited availability of research addressing distributed multi-node communication architectures. The identified research gap provides the motivation for the experimental methodology and evaluation presented in the following chapters.

Chapter 3 – Related Work

Numerous studies have investigated the performance characteristics of real-time operating systems in embedded applications. Most existing research focuses on metrics such as interrupt latency, scheduler overhead, context-switching time, inter-task communication delay, and timing determinism.

Belleza et al. (2018) conducted a performance evaluation of several RTOS platforms for Internet of Things (IoT) devices. Their study compared interrupt latency, scheduler overhead, and overall system performance under identical experimental conditions. The results demonstrated measurable differences among RTOS implementations, highlighting the importance of operating system selection for time-critical embedded applications. Furthermore, the authors emphasized that kernel architecture and

scheduling mechanisms can significantly influence system responsiveness and timing predictability [1].

Mazzi et al. (2021) performed a benchmarking study on ARM Cortex-M4 microcontrollers comparing FreeRTOS and RTX5. Using the ARM Data Watchpoint and Trace (DWT) hardware cycle counter, they measured task-switching time, pre-emption latency, semaphore operations, and inter-task messaging delays. Their results revealed significant differences in scheduling overhead and communication latency between the evaluated RTOS kernels. The study also demonstrated the effectiveness of hardware cycle counters for obtaining high-resolution timing measurements while minimizing measurement overhead. These findings are particularly relevant to the present research, which also employs hardware cycle-counter measurements to evaluate end-to-end communication latency in a distributed RTOS environment [2].

Comparative benchmarking studies have reported measurable performance differences between FreeRTOS and Zephyr when executing common real-time operating system operations. UL Solutions evaluated both RTOS platforms on an ARM Cortex-M4 microcontroller and found that FreeRTOS generally exhibited lower context-switching and interrupt latency, while Zephyr demonstrated better performance for certain synchronization and inter-task communication operations. The authors concluded that the observed differences are primarily related to architectural design choices and kernel implementation strategies rather than indicating that one RTOS is universally superior to the other [3].

Recent benchmarking efforts have continued to investigate RTOS performance across a variety of kernel services. The 2024 RTOS Performance Report evaluated several RTOS kernels using the Thread-Metric benchmark suite, focusing on cooperative scheduling, preemptive scheduling, memory allocation, synchronization processing, and message passing. The study demonstrated that performance differences among RTOS platforms are highly dependent on kernel architecture, implementation strategy, and system configuration. The results further highlighted that no single RTOS consistently outperformed all others across every benchmark category, emphasizing the importance of selecting an RTOS based on application-specific requirements rather than a single performance metric [4], [5].

Ref No	Authors and Year	RTOS / Platform	Evaluation Focus	Key Findings
[1]	Belleza et al. (2018)	Multiple RTOS platforms for IoT devices	Interrupt latency, scheduler overhead, overall system performance	Demonstrated measurable performance differences among RTOS implementations and highlighted the importance of RTOS selection for time-critical applications.
[2]	Mazzi et al. (2021)	FreeRTOS vs. RTX5 on ARM Cortex-M4	Task switching, pre-emption latency, semaphores, inter-task communication	Reported significant differences in scheduling overhead and communication latency. Demonstrated the effectiveness of DWT hardware cycle counters for precise timing analysis.
[3]	S. Huber (2021)	FreeRTOS vs. Zephyr	Context switching, interrupt latency, synchronization operations	FreeRTOS showed lower interrupt and context-switch latency, while Zephyr performed better in some synchronization and communication operations.
[4]	J. Beningo (2024)	Multiple RTOS kernels	Scheduling, synchronization, memory allocation, message passing	Performance varied according to kernel architecture and system configuration. No single RTOS outperformed all others in every category.
[5]	J. Beningo (2024)	Multiple RTOS kernels	RTOS benchmarking methods and evaluation metrics	Highlighted the importance of standardized benchmark methodologies and workload-dependent performance evaluation.
[6]	Promwad (2025)	FreeRTOS, Zephyr, ThreadX	Architectural comparison and RTOS selection criteria	Compared the features, scalability, ecosystem support, and design characteristics of several RTOS platforms and discussed their suitability for different embedded applications.
[7]	Truong et al. (2025)	Azure RTOS ThreadX	Review of kernel architecture and real-time capabilities	Reviewed the architecture, scheduling mechanisms, and real-time characteristics of Azure RTOS ThreadX, highlighting its suitability for deterministic embedded applications.

Table 3.1- Related Work

Table 3.1 summarizes the most relevant studies identified during the literature review. Although previous research has provided valuable insights into RTOS performance, most investigations focus on individual operating systems executing on a single microcontroller. Limited research has examined end-to-end latency in distributed communication architectures involving multiple heterogeneous RTOS platforms. This

research addresses that gap by evaluating communication latency across a multi-node distributed system consisting of Zephyr, FreeRTOS, and Azure RTOS ThreadX operating on identical STM32 hardware platforms.

Additional comparative studies have examined the architectural differences among FreeRTOS, Zephyr, and Azure RTOS ThreadX. FreeRTOS is frequently recognized for its lightweight kernel architecture, low memory footprint, and minimal scheduling overhead, making it suitable for resource-constrained embedded systems. In contrast, Zephyr provides a more comprehensive software ecosystem, including advanced networking capabilities, extensive hardware abstraction layers, and broad hardware support. While these features improve flexibility and scalability, they may introduce additional processing overhead compared with lightweight RTOS kernels. Azure RTOS ThreadX adopts a highly optimized kernel design that emphasizes deterministic scheduling and efficient resource management, making it particularly attractive for time-critical embedded applications [6], [7].

More recently, studies focusing on Azure RTOS ThreadX have reported stable real-time performance and predictable scheduling behaviour. Comparative analyses indicate that ThreadX provides competitive interrupt latency and context-switching performance while maintaining deterministic execution characteristics. These properties have contributed to its adoption in industrial automation, medical devices, aerospace systems, and other safety-critical embedded applications where predictable timing behaviour is essential [7], [8].

Although previous research has provided valuable insights into RTOS performance, most investigations evaluate individual operating systems running on a single microcontroller. Relatively few studies examine distributed communication architectures where signals propagate through multiple nodes executing different RTOS kernels. Consequently, limited information is available regarding the cumulative end-to-end latency introduced by heterogeneous RTOS environments.

The present research addresses this limitation by implementing a distributed communication benchmark consisting of STM32-based nodes running FreeRTOS, Zephyr, and Azure RTOS ThreadX. Unlike previous studies that focus primarily on kernel-level metrics, this work evaluates end-to-end communication latency across a multi-node daisy-chain architecture using hardware cycle-counter measurements, thereby providing a practical assessment of RTOS behaviour in distributed embedded systems.

Chapter 4 – Methodology

4.1 Introduction

This chapter describes the methodology used to evaluate the real-time performance of different Real-Time Operating Systems (RTOSs) within a distributed embedded system. The objective of the study was to compare the latency and timing characteristics of

multiple RTOS implementations when processing and forwarding signals through a communication chain. Unlike traditional benchmarks that evaluate an operating system in isolation, this project investigates the behaviour of RTOSs in a realistic multi-node environment where signals must travel through several processing stages before reaching their destination.

The project was inspired by the growing adoption of heterogeneous embedded systems in industrial automation, Internet of Things (IoT), edge computing, and safety-critical applications. In such systems, different devices may run different operating systems depending on their functionality and resource requirements. Consequently, understanding the latency introduced by each node and the cumulative delay across the entire system is essential for evaluating real-time performance.

The experimental platform consisted of four STM32 Nucleo-F439ZI development boards interconnected through General Purpose Input/Output (GPIO) signals. The boards were arranged in a daisy-chain architecture and executed different RTOS implementations. The final configuration consisted of a Zephyr-based measurement node, a FreeRTOS forwarding node, a second Zephyr forwarding node, and a ThreadX forwarding node. The signal propagated through all boards and was finally returned to the measurement node, where the total end-to-end latency was calculated.

The methodology combines hardware-based timing measurements, interrupt-driven communication, statistical analysis, oscilloscope validation, and graphical visualization using Grafana. This approach enables both quantitative and qualitative evaluation of RTOS behaviour under identical experimental conditions.

4.2 Research Objectives

The primary objective of this study was to compare the real-time performance of multiple RTOS implementations using a common hardware platform and a unified measurement methodology. To achieve this goal, a distributed RTOS communication chain was designed and implemented, enabling the evaluation of signal propagation latency across multiple interconnected RTOS nodes. The study focused on comparing the timing characteristics of FreeRTOS, Zephyr, and ThreadX while investigating key real-time properties such as timing determinism, latency variability, and overall system responsiveness. In addition, software-based latency measurements were validated using external hardware instrumentation to ensure measurement accuracy and reliability. The collected timing data was further analysed and visualized using modern monitoring and visualization tools, allowing trends, distributions, and performance differences to be identified more effectively. Furthermore, the study aimed to identify potential bottlenecks that may arise in heterogeneous embedded systems composed of multiple RTOS platforms. GPIO-based communication was selected as the experimental mechanism because GPIO events represent one of the most fundamental operations in embedded systems and provide a simple, deterministic, and hardware-independent basis for performance comparison.

4.3 Experimental System Overview

The experimental platform was based on four STM32 Nucleo-F439ZI development boards. Each board executed a simple interrupt-driven application responsible for receiving and forwarding a digital signal.

The final system architecture is shown conceptually below:

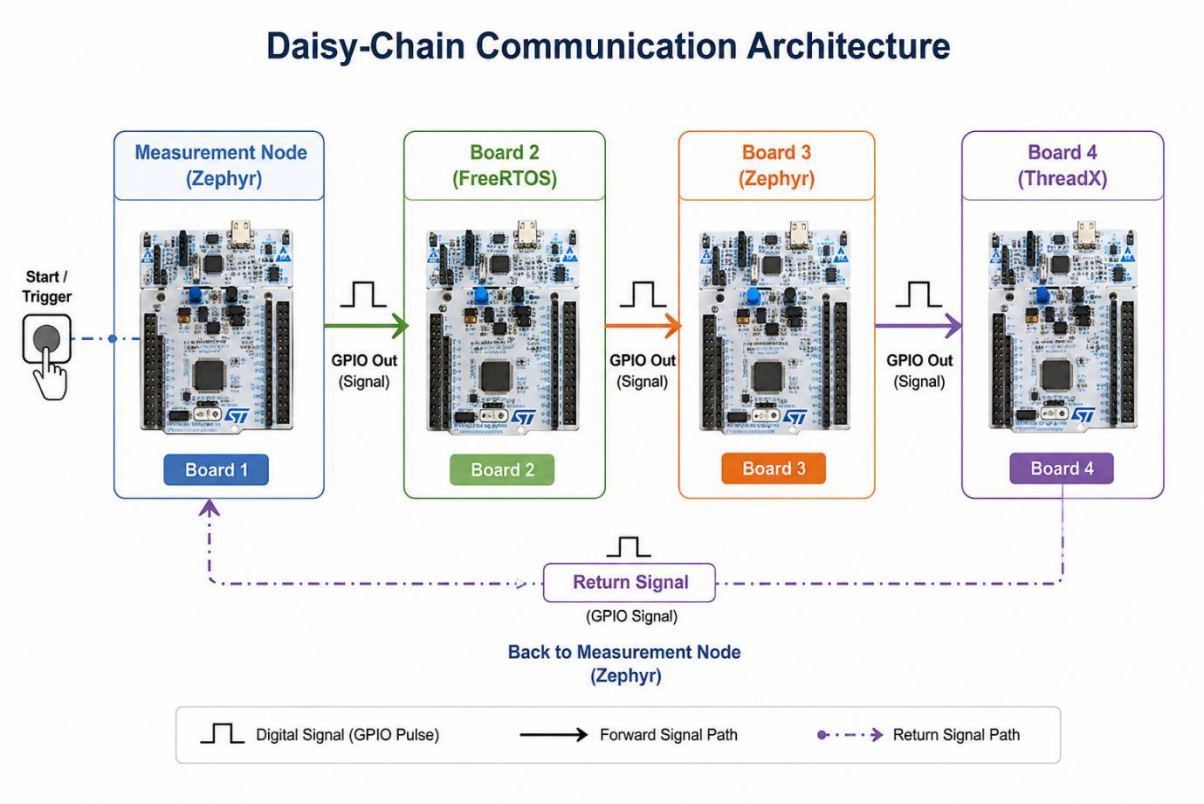

Figure 4.1- Generated by AI

The first board served a dual role within the experimental setup, functioning both as the signal generation node and as the latency measurement node. The remaining boards operated as forwarding nodes, whose primary responsibility was to receive, process, and retransmit the incoming signal to the next node in the communication chain. When the user initiated the benchmark by pressing the start button, the measurement board generated a pulse on its designated GPIO output pin. This pulse was then propagated sequentially through all intermediate nodes, each running a different RTOS configuration. Upon reaching the final node in the chain, the signal was transmitted back to the measurement board through a dedicated return path. The time required for the signal to travel from the measurement node, pass through all forwarding nodes, and return to its origin was recorded as the end-to-end latency of the communication chain. This architecture closely resembles a distributed embedded system in which information must traverse multiple processing stages before reaching its final destination, thereby providing a realistic framework for evaluating RTOS performance in multi-node environments.

4.4 Hardware Platform

4.4.1 Software Development Environment

All software development and firmware deployment activities were performed on a Windows 11 64-bit host computer. STM32CubeIDE was used as the primary development environment for project configuration, compilation, debugging, and firmware deployment. Zephyr applications were developed using the Zephyr SDK and West build system, while FreeRTOS and ThreadX applications were integrated into STM32CubeIDE projects. Firmware images were programmed onto the STM32 Nucleo-F439ZI boards using the onboard ST-Link debugger. Grafana was used for visualization of the collected benchmark data, and CSV files were exported through the serial interface for post-processing and analysis.

Components	Version
Host OS	Windows 11
IDE	STM32 Cube IDE
Zephyr	Dev-4.3.0
FreeRTOS	10.3.1
ThreadX	6.1.7
Compiler	GNU Arm Embedded Toolchain (ARM GCC)
Grafana	13.1.0

Table 4.1- Components

All experiments were conducted using STM32 Nucleo-F439ZI development boards.

The STM32F439ZI microcontroller includes:

- ARM Cortex-M4 core
- Maximum clock frequency of 168 MHz
- Hardware cycle counter support
- GPIO interrupt capability
- Advanced timer peripherals
- Sufficient memory resources for RTOS execution

The use of identical hardware across all experimental nodes eliminated performance variations that could arise from differences in processor architecture, peripheral implementations, or clock frequencies, thereby ensuring a fair comparison between the evaluated RTOS platforms. Each STM32 Nucleo-F439ZI board was powered independently via USB, while all ground connections were electrically tied together to establish a common voltage reference and maintain signal integrity throughout the communication chain. Dedicated GPIO connections were used to link the output pin of one board to the input pin of the subsequent board, forming the daisy-chain architecture employed in the experiments. The use of direct physical wiring provided a realistic communication environment and enabled accurate measurement of signal propagation and processing delays across multiple independent embedded nodes operating under different RTOS configurations.

4.5 RTOS Selection

Three RTOS implementations were used during the final experiments.

4.5.1 Zephyr RTOS

Zephyr RTOS was deployed on two of the four STM32 Nucleo-F439ZI boards used in the experimental platform. Board 1 operated as the measurement and signal generation node, while Board 3 functioned as an intermediate forwarding node within the distributed communication chain. The measurement node was responsible for generating the test signal, recording start and end timestamps using the ARM Cortex-M4 hardware cycle counter, calculating end-to-end latency, and exporting the collected measurement data for subsequent analysis. The Zephyr forwarding node received incoming GPIO interrupts and immediately propagated the signal to the next stage of the communication path. Deploying Zephyr on two separate nodes enabled observation of its behaviour in different positions within the distributed architecture while maintaining identical hardware and communication conditions.

4.5.2 FreeRTOS

FreeRTOS was deployed on Board 2, which served as the first forwarding node in the communication chain. The node received incoming GPIO interrupt events from the measurement board and immediately forwarded the signal to the next node. To minimize application-level overhead, the implementation was intentionally kept lightweight and performed only the essential signal forwarding operation. This design ensured that the measured latency primarily reflected interrupt handling and RTOS behaviour rather than application-specific processing.

4.5.3 Azure RTOS ThreadX

Azure RTOS ThreadX was deployed on Board 4, which acted as the final forwarding node in the communication chain before returning the signal to the measurement board. Similar to the other forwarding nodes, the implementation was intentionally limited to interrupt-driven signal forwarding in order to minimize software overhead. The ThreadX node received the incoming signal, generated the corresponding output pulse, and transmitted the signal back to the measurement node, allowing the total end-to-end communication latency of the distributed architecture to be measured.

4.6 Communication Architecture

The communication architecture was based on interrupt-driven GPIO signalling, allowing each node to react immediately to incoming events without relying on continuous polling. Each node in the communication chain consisted of a single GPIO input pin, a single GPIO output pin, and an interrupt callback function responsible for handling incoming signals. When a rising edge was detected on the input pin, the corresponding Interrupt Service Routine (ISR) was triggered automatically. The ISR executed a minimal forwarding routine, generating a pulse on the output pin and transmitting the signal to the next node in the chain. This interrupt-driven approach reduced processor overhead, improved responsiveness, and more closely reflected the event-driven behaviour commonly found in real-world embedded

systems. The communication sequence began when the measurement node generated a GPIO pulse, which was subsequently received, processed, and forwarded by each intermediate node until it reached the final node. The final node then returned the signal to the measurement board, completing the communication cycle and enabling measurement of the total end-to-end latency introduced by the distributed system.

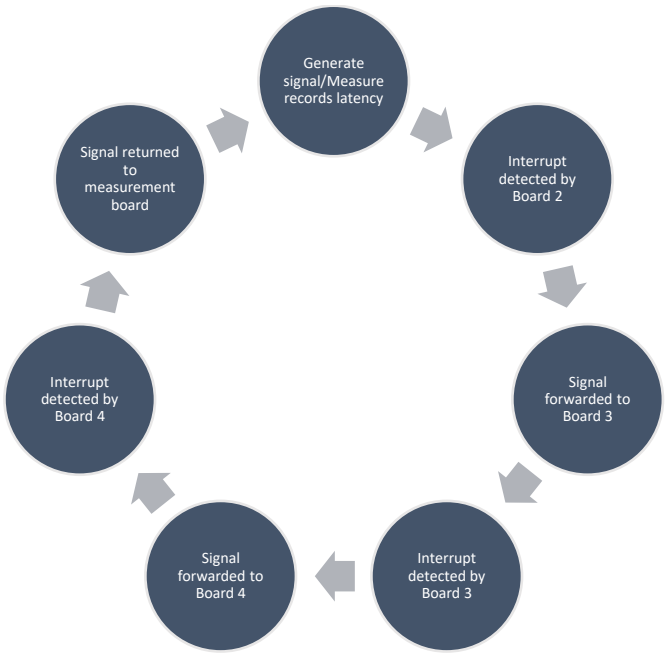

Figure 4.2- Architecture of RTOS-Snake

This architecture closely resembles event-driven industrial systems where external events trigger immediate responses.

4.6.1 Measurement Node Benchmark Procedure

To provide a clearer representation of the benchmark execution workflow, Algorithm 4.1 summarizes the operational sequence implemented on the Zephyr-based measurement node. The algorithm illustrates the initialization process, signal generation procedure, latency measurement mechanism, and data collection workflow used throughout the experiments. After system initialization, a set of preliminary signal transmissions is executed to stabilize the communication chain and eliminate startup-related effects. The benchmark then enters the measurement phase, during which latency values are collected using the hardware cycle counter and stored for subsequent statistical analysis and visualization.

A short inter-sample delay of 50ms was inserted between consecutive measurements to allow the communication chain to stabilize before the next benchmark iteration.

```
Algorithm 1: Measurement Node Benchmark Procedure

1: Initialize GPIO interfaces and interrupt handlers

2: Initialize hardware cycle counter

3: Wait for user button press

4: Execute warm-up phase
for i ← 1 to 20 do
Generate signal pulse
Wait for returned signal
end for

5: Reset measurement counters

6: Execute benchmark phase
for i ← 1 to 5000 do

    Record start timestamp

    Generate output pulse

    Wait for returned signal

    Record end timestamp

    Calculate latency

    Store latency value

  end for

7: Calculate statistical metrics
• Minimum Latency
• Maximum Latency
• Average Latency
• Latency Range

8: Export collected measurements to CSV format

9: Display benchmark results
```

Algorithm 4.1. Measurement node benchmark procedure used for latency measurement and data collection

As illustrated in Algorithm 4.1, the benchmark execution consists of three primary stages: initialization, measurement acquisition, and result processing. During initialization, the GPIO interfaces, interrupt handlers, and hardware timing mechanisms are configured. The measurement phase repeatedly generates a signal pulse, records timestamps using the processor cycle counter, waits for the returned signal, and calculates the corresponding end-to-end latency. The measured values are stored in memory and the procedure is repeated until the predefined number of samples has been collected. Finally, statistical metrics are calculated and the complete dataset is exported in CSV format for further analysis and visualization using Grafana.

4.6.2 FreeRTOS Forwarding Node Operation

The FreeRTOS forwarding node was responsible for receiving an incoming GPIO signal and propagating it to the next stage of the communication chain. Similar to the other forwarding nodes, the implementation was intentionally kept lightweight in order to minimize application-level overhead and ensure that the measured latency predominantly reflected RTOS behaviour and interrupt processing performance. The node was configured with a GPIO input pin operating in interrupt mode and multiple GPIO output pins used for signal forwarding and debugging purposes. When a rising edge was detected on the input pin, an external interrupt was generated and the corresponding callback routine was executed. The interrupt handler immediately generated output pulses on the designated GPIO pins, thereby forwarding the received signal to the next node in the communication path. Algorithm 4.3 summarizes the initialization and interrupt-driven signal forwarding procedure implemented on the FreeRTOS node.

```
Algorithm 4.2: FreeRTOS Forwarding Node Procedure

Initialize HAL library

Configure system clock

Initialize GPIO interfaces

Configure GPIO interrupt

Create default RTOS task

Start FreeRTOS scheduler

Wait for incoming signal

Upon rising-edge interrupt:

    Generate output pulse on forwarding pins

    Activate debug indicator

    Maintain pulse duration

    Reset output pins

    Return from interrupt

Repeat indefinitely
```

Algorithm 4.2. FreeRTOS forwarding node initialization and interrupt-driven signal forwarding procedure

As illustrated in Algorithm 4.2, the forwarding node remains idle after system initialization and relies entirely on interrupt-driven operation for signal processing. Upon reception of an incoming signal, the interrupt service routine generates the corresponding output pulse and immediately returns control to the operating system. This event-driven design minimizes processor utilization while providing rapid signal

propagation and predictable timing behaviour. By restricting the interrupt routine to only the essential forwarding operation, software-induced latency is reduced and a more accurate evaluation of RTOS performance is achieved.

4.6.3 Zephyr Forwarding Node Operation

The Zephyr forwarding node was implemented as an interrupt-driven GPIO forwarding application. This node received the incoming signal on PA4 and forwarded it simultaneously through two output pins, PA3 and PB1. In addition to signal forwarding, the onboard LD1 indicator was used as a visual debugging mechanism to confirm signal activity during operation. Unlike the measurement node, this forwarding node did not perform latency calculation or data logging. Its purpose was limited to detecting incoming GPIO transitions and immediately updating the output pins according to the input state. The interrupt was configured to detect both rising and falling edges, allowing the outputs and the LED to be activated when the input signal was high and deactivated when the signal returned low. Algorithm 4.4 summarizes the operation of the Zephyr forwarding node.

```
Algorithm 4.3: Zephyr Forwarding Node Procedure

Initialize GPIO devices

Verify GPIO device readiness

Configure PA4 as input

Configure PA3 as output

Configure PB1 as output

Configure LD1 as debug LED output

Configure GPIO interrupt on both rising and falling edges

Register interrupt callback function

Wait indefinitely for input signal

Upon GPIO interrupt:

    Read input state on PA4

    if input state is HIGH then

        Set PA3 HIGH

        Set PB1 HIGH

        Turn LD1 ON

    else

        Set PA3 LOW

        Set PB1 LOW

        Turn LD1 OFF

    end if

Return from interrupt

Repeat indefinitely
```

Algorithm 4.4. Zephyr forwarding node interrupt-driven GPIO forwarding procedure

As shown in Algorithm 4.4, the Zephyr forwarding node follows a simple event-driven execution model. After initialization, the node remains in a low-power waiting state until an interrupt is generated by a change on the input pin. When the input signal becomes active, the node sets both forwarding outputs and turns on the debug LED. When the input signal becomes inactive, the outputs and LED are cleared. This design ensures that the forwarded signal follows the input state while keeping the interrupt routine minimal and avoiding unnecessary processing inside the application.

4.6.4 ThreadX Forwarding Node Initialization and Interrupt Handling

The forwarding nodes were designed to perform a minimal set of operations in order to minimize application-level overhead and ensure that the measured latency primarily reflected RTOS behaviour and interrupt processing performance. Each forwarding node was configured with a single GPIO input pin, a single GPIO output pin, and an interrupt service routine responsible for handling incoming signals. Upon detection of a rising edge on the input pin, the corresponding interrupt was triggered automatically. The interrupt handler immediately generated an output pulse and forwarded the signal to the next node in the communication chain without performing any additional computation or processing. This lightweight implementation reduced software-induced delays and provided a consistent forwarding mechanism across all RTOS platforms evaluated in this study. Algorithm 4.2 summarizes the initialization and signal-forwarding procedure implemented on the forwarding nodes.

```
Algorithm X: ThreadX Forwarding Node Initialization and Interrupt Handling

1: Start program

2: Initialize GPIO and EXTI configuration

3: Enable GPIOA, GPIOB, and SYSCFG peripheral clocks

4: Configure PA4 as GPIO input

5: Configure PA3 as GPIO output

6: Configure PB0, PB7, and PB14 as LED output pins

7: Set PA3 to LOW state

8: Turn off all LEDs

9: Configure EXTI line 4 to use PA4 as interrupt source

10: Configure EXTI4 to trigger on rising edge

11: Clear any pending EXTI4 interrupt flag

12: Set EXTI4 interrupt priority

13: Enable EXTI4 interrupt in NVIC

14: Start ThreadX kernel

15: Wait indefinitely for incoming signal interrupt

16: When a rising edge is detected on PA4:

        Trigger EXTI4 interrupt

        Execute interrupt handler

        Generate output signal on PA3

        Optionally update LED status

        Clear interrupt pending flag

        Return from interrupt

17: Repeat indefinitely
```

Algorithm 4.2. Forwarding node initialization and interrupt-driven signal forwarding procedure

As illustrated in Algorithm 4.2, the forwarding node remains in an idle state after initialization and consumes processing resources only when an external event occurs. The interrupt-driven design enables rapid signal propagation while minimizing processor utilization and timing variability. By restricting the interrupt service routine to only the essential forwarding operation, the influence of application-specific software on the latency measurements was minimized, resulting in a more accurate evaluation of RTOS behaviour within the distributed communication chain.

4.7 Timing Measurement Method

Accurate timing measurement is a fundamental requirement when evaluating the performance of real-time operating systems, as even small timing variations can significantly affect system behaviour in latency-sensitive applications. To achieve high-resolution measurements, the hardware cycle counter available on the STM32F439ZI microcontroller was utilized. Unlike software-based timers, which may be affected by operating system scheduling and timer resolution limitations, the hardware cycle counter increments once for every CPU clock cycle and therefore provides precise cycle-level timing information. At the beginning of each measurement cycle, a timestamp was captured immediately before transmitting the GPIO signal from the measurement node. A second timestamp was then recorded when the returning signal was received after traversing the entire communication chain. The end-to-end latency was calculated as the difference between these two timestamps, as shown in Equation X:

$$\mathbf{Latency_{cycles} = T_{end} - T_{start}}$$

- $\mathbf{T_{start}}$**:** Hardware cycle counter value recorded at the instant the signal is transmitted
- $\mathbf{T_{end}}$**:** Hardware cycle counter value recorded at the instant the response signal is received.

Since the STM32F439ZI microcontroller operated at a clock frequency of 168 MHz, each clock cycle corresponded to approximately 5.95 nanoseconds, enabling highly accurate latency measurements. The use of hardware cycle counting minimized measurement overhead, reduced software-induced timing inaccuracies, and improved the repeatability and reliability of the experimental results.

A pulse width of 100 μs was used to ensure reliable signal detection across all forwarding nodes and to prevent missed interrupt events.

4.7.1 Cycle-to-Time Conversion

The STM32F439ZI microcontroller used in this study operates at a clock frequency of:

$$f = 168 \text{ MHz}$$

Since:

$$1\ \text{MHz} = 10^6\ \text{Hz}$$

the processor frequency can be expressed as:

$$f = 168 \times 10^6\ \text{Hz}$$

The duration of a single CPU clock cycle is obtained by taking the inverse of the clock frequency:

$$T_{cycle} = \frac{1}{f}$$

Substituting the processor frequency into the equation:

$$T_{cycle} = \frac{1}{168 \times 10^6}$$

$$T_{cycle} = 5.95238 \times 10^{-9}\ \text{s}$$

Therefore:

$$T_{cycle} \approx 5.95\ \text{ns}$$

This indicates that each increment of the hardware cycle counter corresponds to approximately **5.95 nanoseconds**.

To convert measured cycle counts into real-time values, the following equation was used:

$$Latency_{time} = Latency_{cycles} \times T_{cycle}$$

Alternatively, the conversion can be expressed as:

$$Latency_{time} = \frac{Latency_{cycles}}{f}$$

4.8 Interrupt-Driven Design

Polling mechanisms were intentionally avoided throughout the experimental implementation because they require the processor to continuously monitor the state of an input signal, resulting in unnecessary CPU utilization and potentially increasing timing variability. Instead, GPIO interrupts were employed to detect incoming signals, allowing the processor to remain idle until an event occurred. This interrupt-driven approach offers several advantages, including reduced processor overhead, faster event detection, improved timing determinism, and a closer representation of the event-driven behaviour commonly found in industrial real-time systems. Furthermore, the Interrupt Service Routine (ISR) on each node was intentionally kept as simple as possible. Upon receiving an incoming signal, the ISR immediately generated the corresponding output pulse and returned control to the operating system without performing any additional processing. This design minimized the influence of application-level software on the measurements and ensured that the recorded latency primarily reflected the behaviour of the RTOS, interrupt handling mechanisms, and communication architecture rather than application-specific overhead.

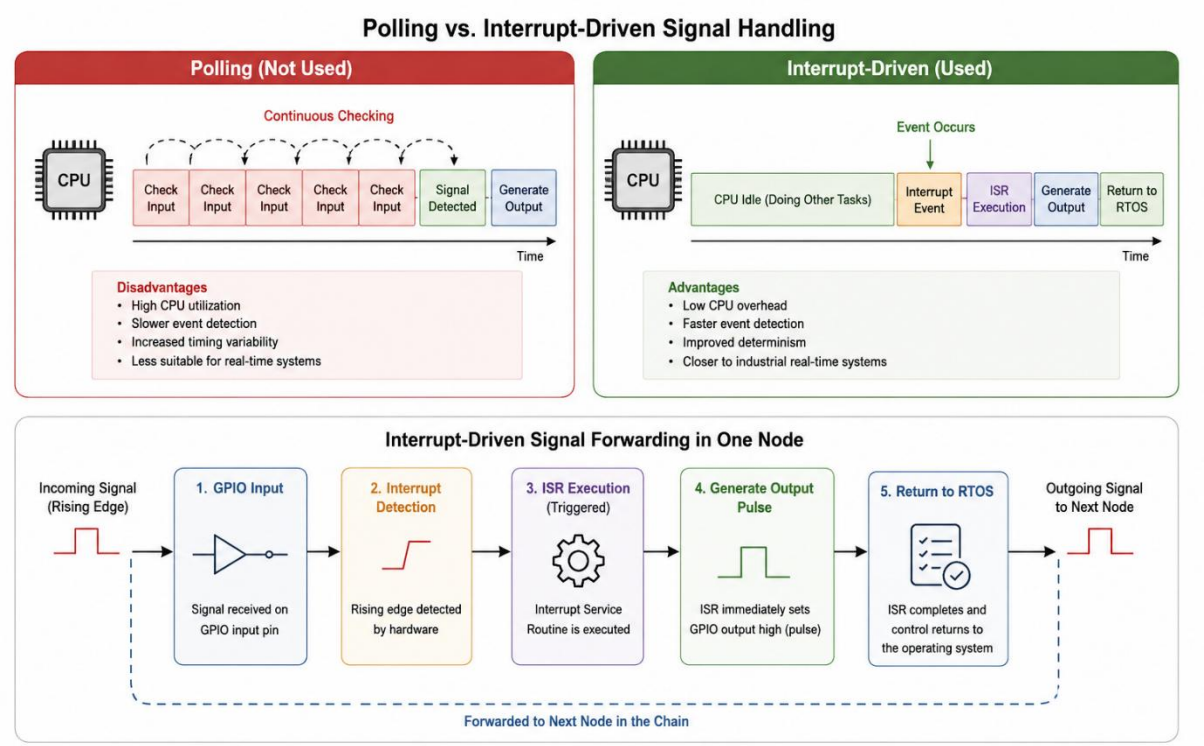

Figure 4.3- Generated by AI

Figure 4.3 Comparison of polling-based and interrupt-driven signal handling**.** The figure illustrates the fundamental differences between polling and interrupt-driven communication mechanisms in embedded

systems. While polling continuously checks the state of an input signal and consumes processor resources even when no event occurs, the interrupt-driven approach allows the CPU to remain idle until a signal is detected. Upon detection of a rising edge, an interrupt is generated, the Interrupt Service Routine (ISR) is executed, and an output pulse is immediately forwarded to the next node in the communication chain. This approach reduces CPU overhead, improves responsiveness and timing determinism, and more accurately represents the behaviour of real-time industrial embedded systems. The lower section of the figure demonstrates the signal forwarding process implemented on each RTOS node used in the experimental setup.

4.9 Benchmark Procedure

The benchmark followed a structured execution sequence to ensure consistency, repeatability, and reliability across all experiments. Initially, all STM32 Nucleo-F439ZI boards were powered on and configured with their respective RTOS implementations. During the system initialization phase, the GPIO peripherals, interrupt handlers, communication interfaces, and measurement mechanisms were configured and verified to ensure proper operation. Once the initialization process was completed, the benchmark was started manually by pressing the onboard push button located on the Zephyr-based measurement node, which triggered the beginning of the signal propagation experiment.

Before collecting any measurement data, a warm-up phase was executed to eliminate startup-related effects and stabilize the behaviour of the system. During this phase, twenty preliminary signal transmissions were performed without recording latency values. The purpose of these transmissions was to ensure that all interrupt handlers, communication paths, and RTOS components were operating under steady-state conditions before the actual benchmark measurements began. This approach reduced the influence of initialization artifacts and improved the consistency of the collected data.

Following the warm-up phase, the benchmark entered the measurement phase. For each iteration, the measurement node generated an output pulse and transmitted it through the communication chain. The system then waited for the corresponding return signal, after which the latency was calculated using the hardware cycle counter and stored in memory. This process was repeated until the predefined number of samples had been collected. Upon completion of the benchmark, statistical calculations were performed on the recorded dataset, and the collected measurements were exported through the serial interface in CSV format for further analysis, visualization, and comparison of the different RTOS implementations.

4.10 Sample Collection Strategy

A total of 5,000 measurements were collected for each experiment.

Using a large sample size provides several benefits:

- Increased statistical confidence.
- Improved representation of system behaviour.
- Better identification of outliers.
- More reliable average values.

A small number of measurements could easily be affected by random variations. By collecting thousands of samples, the resulting dataset more accurately represents the true behaviour of the system.

4.11 Data Logging

Each latency measurement was recorded as a structured data entry consisting of a sample identifier and the corresponding latency value expressed in CPU cycles. The collected measurements were exported through the serial interface and stored in CSV format to ensure portability and compatibility with data analysis tools. Separate datasets were generated for each communication path and RTOS configuration, enabling independent evaluation of the experimental scenarios. These datasets formed the basis for subsequent statistical analysis, visualization, and performance comparison. The use of a standardized CSV format simplified data management and facilitated integration with Grafana for dashboard creation and graphical exploration of latency characteristics.

Samples	Latency
1	value
2	value
3	value
… … …	… … …
5000	value

Table 4.2- Path result format

4.12 Statistical Analysis

After completion of the benchmark, several statistical metrics were calculated.

Additional graphical analysis was performed using histograms and time-series visualizations to investigate latency distributions and temporal behaviour

Metric	Description	Formula / Definition
Minimum Latency	Represents the fastest observed response during the experiment.	Smallest latency value recorded among all samples.
Maximum Latency	Represents the slowest observed response during the experiment.	Largest latency value recorded among all samples.
Average Latency	Represents the mean latency across all collected measurements.	$\text{Average Latency} = \frac{\sum_{i=1}^{N} \text{Latency}_i}{N}$
Latency Range	Represents the spread between the fastest and slowest measurements and provides an indication of timing variability.	Latency Range=Maximum Latency−Minimum Latency

Table 4.3- Statistical analysis

4.13 Data Visualization Using Grafana

Grafana was employed as the primary visualization and analysis platform for interpreting the collected latency measurements. After the benchmark execution, the exported CSV datasets were imported into Grafana and displayed through multiple dashboard panels designed to highlight different aspects of system performance. These visualizations included time-series plots for observing latency trends over the 5,000 collected samples, histograms for analysing latency distributions, statistical summary panels for presenting key metrics such as minimum, maximum, average latency, and latency range, as well as comparative views for evaluating the performance of different RTOS implementations. By transforming large volumes of raw numerical data into intuitive graphical representations, Grafana enabled rapid identification of timing patterns, outliers, and variability characteristics that would have been difficult to detect through manual inspection of the datasets alone.

4.14 Oscilloscope Validation

To validate the accuracy of the software-based latency measurements, an external oscilloscope was employed as an independent verification tool. The oscilloscope probes were connected to the signal transmission pin on the measurement node and the

corresponding signal return pin where the propagated signal re-entered the system. By capturing both waveforms simultaneously, the time difference between their rising edges could be measured directly in hardware. This experimentally observed delay was then compared with the latency values obtained from the software cycle counter measurements. The close agreement between the oscilloscope readings and the software-generated results provided strong evidence that the implemented measurement methodology accurately represented the actual physical behaviour of the communication chain and that the reported latency values were reliable.

4.15 Reliability and Repeatability

To ensure the reliability and fairness of the experimental results, several measures were implemented throughout the benchmarking process. All experiments were conducted using the same hardware platform, operating at an identical clock frequency and utilizing a consistent GPIO configuration across all boards. Furthermore, the physical wiring topology remained unchanged for every test, ensuring that each RTOS was evaluated under the same communication conditions. A fixed benchmark procedure was followed for all measurements, including identical signal generation, forwarding, and latency recording mechanisms. In addition, a large sample size of 5,000 measurements was collected for each experiment to improve statistical significance and reduce the influence of random fluctuations or outliers. By maintaining identical experimental conditions throughout the study, measurement bias was minimized and the resulting comparisons became more reliable and reproducible.

4.16 Limitations

Despite careful experimental design and implementation, several limitations should be acknowledged. First, the measurements obtained in this study represent the overall end-to-end latency of the communication chain rather than the intrinsic kernel latency of each RTOS. Consequently, the recorded values include not only operating system behaviour but also the cumulative effects of signal transmission through multiple nodes. Second, factors such as GPIO propagation delay, interrupt service routine execution time, context switching overhead, and software processing contribute to the measured latency and cannot be completely isolated from the RTOS itself. Third, all experiments were conducted on a single hardware platform, namely the STM32F439ZI microcontroller, which may limit the generalizability of the results to other processor architectures or hardware configurations. Nevertheless, these limitations do not diminish the value of the study, as the objective was to evaluate RTOS performance within a realistic distributed embedded environment. Therefore, the proposed methodology provides a practical and representative assessment of end-to-end latency and timing behaviour in heterogeneous embedded systems.

4.17 Chapter Summary

This chapter presented the methodology used to compare the performance of Zephyr, FreeRTOS, and ThreadX within a distributed embedded communication chain. The experimental platform consisted of four STM32 Nucleo-F439ZI development boards interconnected through dedicated GPIO connections and arranged in a daisy-chain architecture. A signal generated by the measurement node propagated sequentially through each RTOS-based node before being returned to the originating board, allowing the total end-to-end latency of the system to be measured. To achieve high timing accuracy, hardware cycle counting was employed using the STM32 processor's built-in cycle counter, providing cycle-level resolution for all latency measurements. The communication mechanism was implemented using interrupt-driven GPIO signalling to emulate the event-driven behaviour commonly found in industrial and real-time embedded systems.

To ensure statistical significance and measurement reliability, a large dataset consisting of 5,000 samples was collected for each experimental scenario. The recorded data was exported in CSV format and analysed using multiple statistical metrics, including minimum latency, maximum latency, average latency, and latency range. Furthermore, Grafana was utilized to visualize the collected data through time-series graphs, histograms, and comparative dashboards, enabling a more comprehensive interpretation of timing behaviour and system performance. In addition to software-based measurements, an external oscilloscope was used to validate the accuracy of the timing results by directly measuring the propagation delay between transmitted and returned signals. This hardware validation provided confidence that the software-generated measurements accurately reflected the actual physical behaviour of the system.

Several measures were implemented throughout the experimental process to ensure fairness, repeatability, and consistency. All RTOS implementations were evaluated using identical hardware, common clock frequencies, equivalent GPIO configurations, and the same communication topology. The benchmark procedure, signal generation mechanism, and measurement methodology were also kept constant across all experiments to minimize measurement bias and ensure a fair comparison between operating systems. Although the study focused on a single microcontroller platform and measured end-to-end system latency rather than isolated kernel latency, the methodology provides a realistic representation of how different RTOSs behave in practical distributed embedded systems. Consequently, the results obtained from this methodology offer valuable insights into the latency characteristics, determinism, and interoperability of multiple RTOS platforms operating within a heterogeneous embedded environment.

Chapter 5 – Evaluation and Discussion

5.1 Introduction

This chapter presents the experimental results obtained from the distributed RTOS communication benchmark described in the previous chapter. The objective of the evaluation was to compare the latency characteristics of FreeRTOS, Zephyr, and ThreadX under identical hardware and measurement conditions. A total of 5,000 latency samples were collected for each communication path using the hardware cycle counter available on the STM32F439ZI microcontroller. The collected datasets were exported in CSV format and analysed using statistical methods and graphical visualization techniques implemented in Grafana. The evaluation focuses on latency behaviour, timing variability, determinism, and overall communication performance across the different RTOS configurations.

5.2 Experimental Results

Three independent benchmark scenarios were evaluated. In each scenario, the measurement node executed the same benchmark procedure while the communication path included a different forwarding node implementation. Path A evaluated the FreeRTOS forwarding node, Path B evaluated the Zephyr forwarding node, and Path C evaluated the ThreadX forwarding node. For each path, 5,000 latency measurements were collected after the completion of the warm-up phase. The recorded values represent the total end-to-end latency required for a signal to propagate through the communication chain and return to the measurement node.

The collected datasets were processed to determine the minimum latency, maximum latency, average latency, and latency range. These metrics provide an overview of both performance and timing stability. In addition, time-series plots, histograms, and statistical summary panels were generated using Grafana to facilitate detailed analysis of latency behaviour and variability.

Path	RTOS	Samples	Minimum (cycles)	Maximum (cycles)	Average (cycles)	Range (cycles)
Path A	Measuring Zephyr+ FreeRTOS	5000	1435	2560	1456	1125
Path B	Measuring Zephyr+ FreeRTOS+ Zephyr	5000	2336	3572	2384	1236
Path C	Measuring Zephyr+ FreeRTOS+Zephyr+ThreadX	5000	2488	4048	2585	1560

Table 5.1- Summary of latency measurements obtained from the three RTOS communication paths.

5.2.1 Conversion of Cycle Counts to Time Units

Since the hardware cycle counter reports latency in CPU cycles, the measured values can be converted into time units using the processor clock frequency. The STM32F439ZI microcontroller operates at a clock frequency of 168 MHz, corresponding to a clock period of approximately 5.95ns per cycle.

For an average measured latency of path C:

$$Latency_{cycles} = 2585\ cycles$$

the corresponding time value is:

$$\boldsymbol{Latency_{time}} = \frac{2585}{168 \times 10^6}$$

$$\boldsymbol{Latency_{time}} = 15.38\ \mu s$$

$$\boldsymbol{Latency_{time}} = 15386\ ns$$

Thus, an average latency of **2585 cycles** correspond to approximately **15.38** μs for the end-to-end communication path evaluated in Path C.

5.2.2 Path A – Measuring Zephyr + FreeRTOS Results

Path A evaluated the first communication scenario in the proposed distributed architecture. In this configuration, the signal was generated by the Zephyr-based measurement node, forwarded through the FreeRTOS node, and subsequently returned to the measurement board. The measured latency therefore represents the complete round-trip communication delay associated with signal propagation between the measurement node and the FreeRTOS forwarding node.

Following completion of the warm-up phase, a total of 5,000 latency samples were collected using the hardware cycle counter. The measured latency values ranged from 1435cycles to 2560 cycles, resulting in a latency range of 1125 cycles. The average measured latency was 1456 cycles, corresponding to approximately 8.66 μs when converted using the 168 MHz processor clock frequency.

The time-series visualization showed a stable latency profile throughout the experiment, with the majority of measurements remaining close to the average value. No significant outliers or abnormal timing spikes were observed within the collected dataset. Figure X presents the latency distribution for Path A, while Table 5.1 summarizes the corresponding statistical metrics.

The reported latency values include the cumulative effects of GPIO interrupt handling, interrupt service routine execution, signal propagation between interconnected boards, and software processing within the communication chain. Consequently, the measured results represent the end-to-end latency of the evaluated communication path.

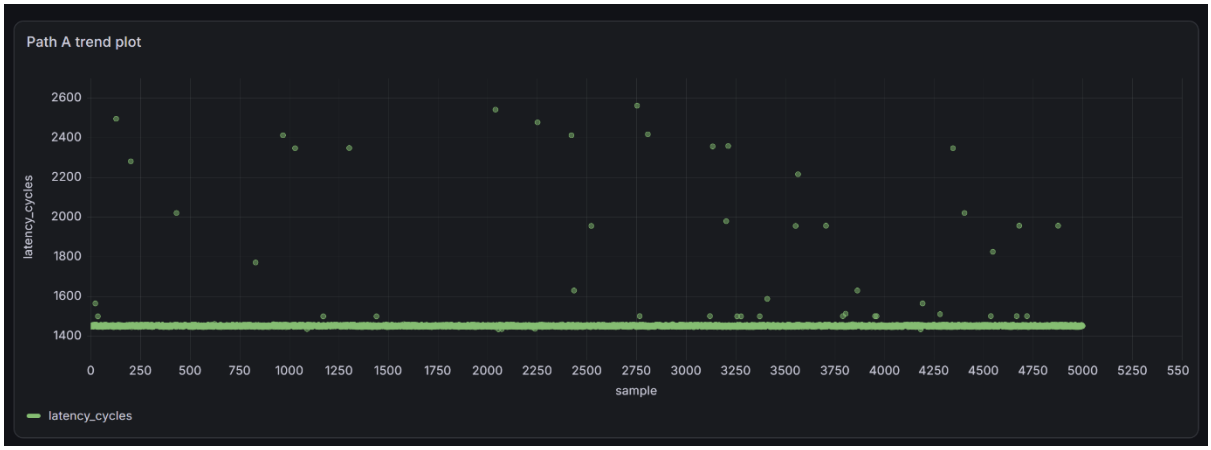

Figure 5.1 – Latency trend plot for Path A

The trend plot shows that most Path A samples remain concentrated around 1450 cycles, indicating stable timing behavior. A small number of isolated latency spikes are visible, but they occur infrequently compared with the total number of collected samples.

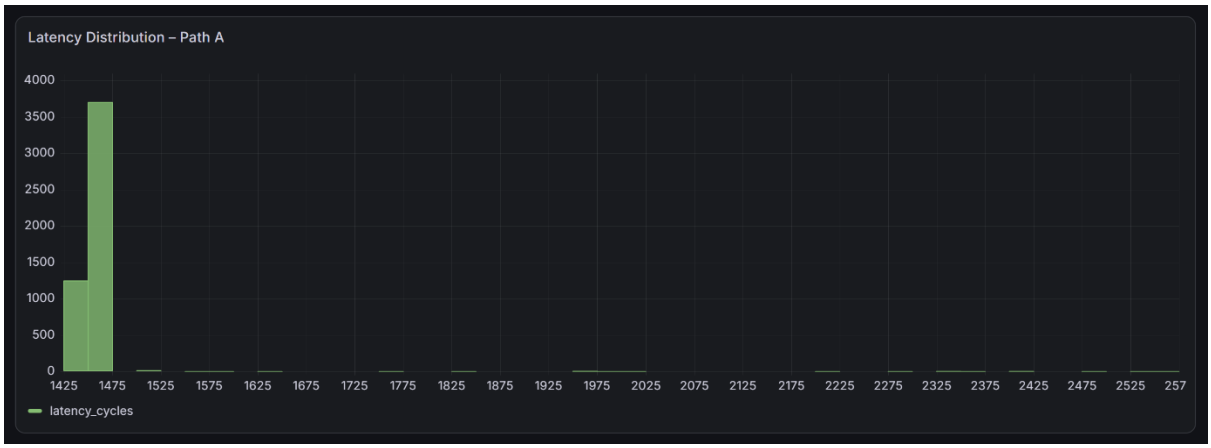

Figure 5.2 – Latency distribution histogram for Path A

5.2.3 Path B – Measuring Zephyr + FreeRTOS + Zephyr Results

Path B evaluated the distributed communication chain in which the transmitted signal propagated through both the FreeRTOS and Zephyr forwarding nodes before returning to the measurement node. In this configuration, the signal was generated by the Zephyr-based measurement board, forwarded through the FreeRTOS node, propagated to the Zephyr forwarding node, and subsequently returned to the measurement board. Consequently, the measured latency represents the cumulative delay introduced by multiple processing stages operating under different RTOS environments.

Using the benchmark methodology described in Chapter 4, a total of 5,000 latency samples were collected under identical hardware and software conditions. The minimum recorded latency was 2336 cycles, while the maximum latency reached 3572

cycles. This resulted in a latency range of 1236 cycles, with an average measured latency of 2384 cycles, corresponding to approximately 14.19 µs when converted using the 168 MHz processor clock frequency.

The time-series visualization indicated a stable latency distribution throughout the benchmark execution. Most measurements remained concentrated around the average value, with only minor fluctuations observed across the collected dataset. No significant outliers or abnormal timing spikes were detected during the experiment. Figure X presents the latency distribution for Path B, while Table 5.1 summarizes the corresponding statistical metrics.

The measured latency includes the combined effects of GPIO interrupt handling, interrupt service routine execution, signal propagation between boards, and software processing overhead introduced by the forwarding nodes. Therefore, the reported latency values include the cumulative effects of GPIO interrupt handling, interrupt service routine execution, signal propagation between interconnected boards, and software processing within the communication chain.

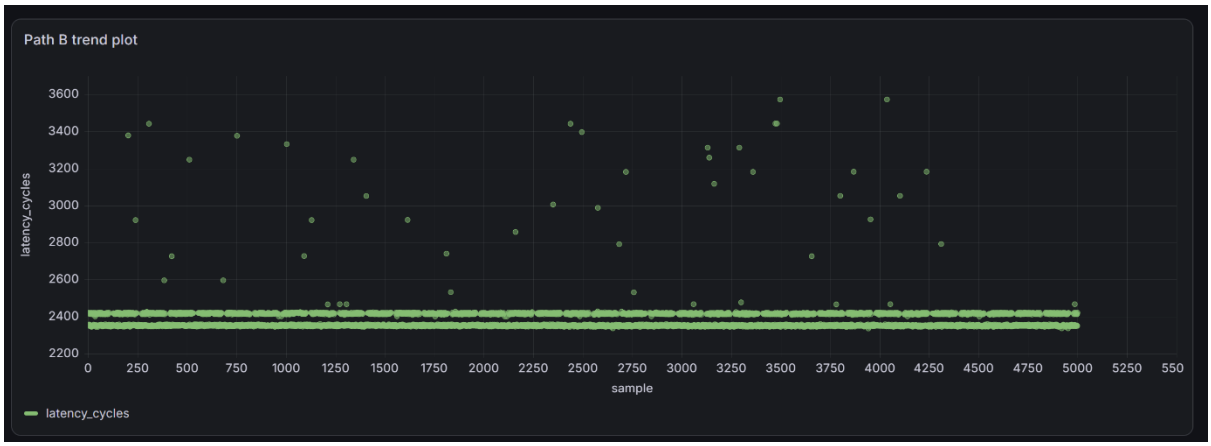

Figure 5.3 – Latency trend plot for Path B

The trend plot for Path B shows that most latency values remain concentrated around 2400 cycles. The increase compared with Path A is expected because Path B includes an additional forwarding stage in the daisy-chain communication path.

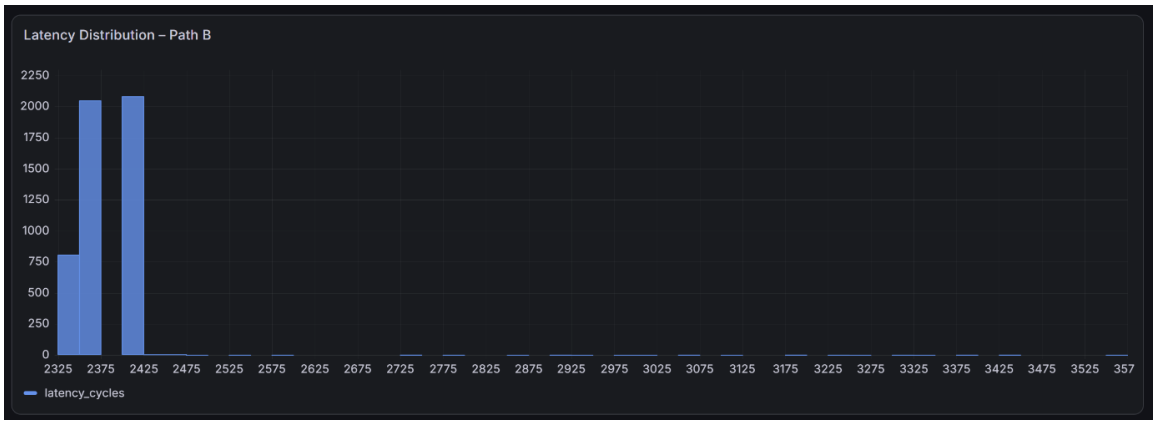

Figure 5.4 – Latency distribution histogram for Path B.

5.2.4 Path C – Measuring Zephyr + Free RTOS + Zephyr + ThreadX Results

Path C evaluated the complete distributed communication chain in which the transmitted signal propagated sequentially through three forwarding nodes before returning to the measurement node. In this configuration, the signal originated from the Zephyr-based measurement node, passed through the FreeRTOS forwarding node, continued through the Zephyr forwarding node, reached the ThreadX forwarding node, and was finally returned to the measurement board. Consequently, the measured latency represents the cumulative delay introduced by all processing stages within the communication chain.

After completion of the warm-up procedure, 5,000 benchmark iterations were executed and the corresponding latency values were recorded. The minimum observed latency was 2488 cycles, whereas the maximum latency reached 4048 cycles. The average latency measured across the entire dataset was 2585 cycles, corresponding to approximately 15.38 µs. The resulting latency range was calculated as 1560cycles.

The time-series visualization showed a stable latency distribution throughout the experiment. The majority of the collected measurements remained concentrated around the average value, with only minor fluctuations observed across the dataset. No significant outliers or abnormal timing spikes were detected during the benchmark execution. Figure X presents the latency distribution for Path C, while Table 5.1 summarizes the corresponding statistical metrics.

The reported values correspond to the end-to-end latency measured across the complete communication chain. The measurements include all delays introduced during signal transmission, forwarding, and reception throughout the evaluated communication path.

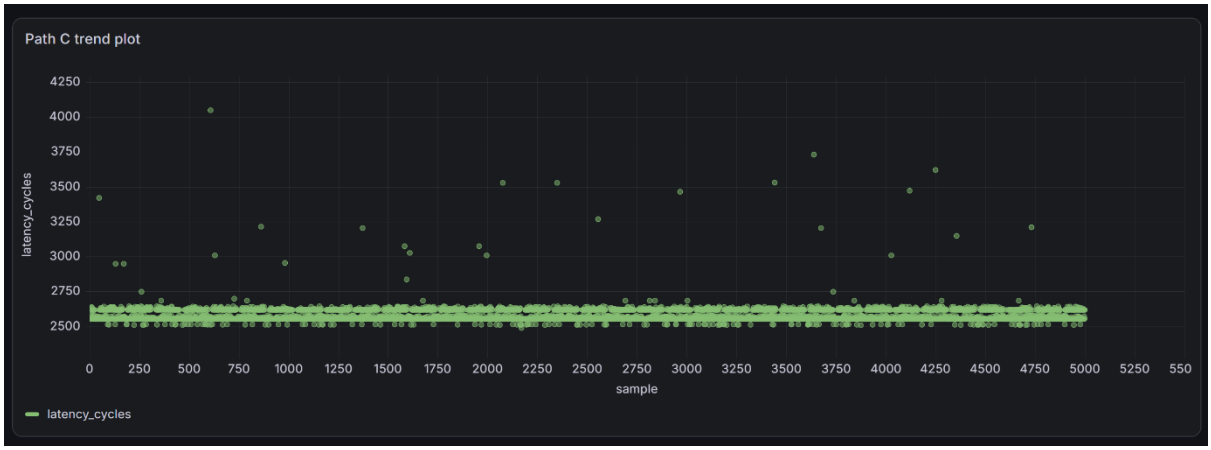

Figure 5.5 – Latency trend plot for Path C

The trend plot for Path C shows a stable latency profile around 2550–2650 cycles. Occasional higher latency values are observed, but the majority of samples remain close to the average latency.

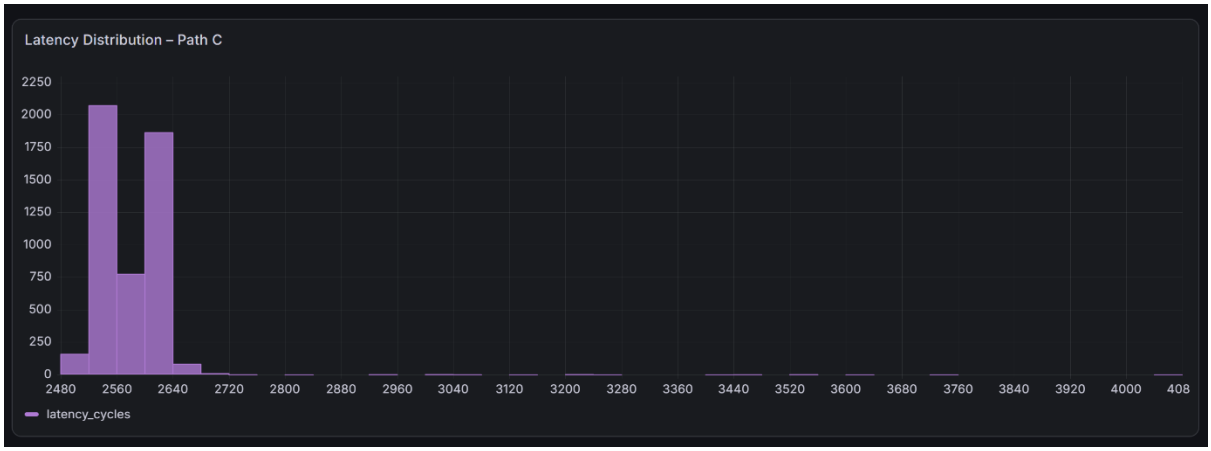

Figure 5.6 – Latency distribution histogram for Path C

5.2.5 Comparison of Average End-to-End Latency

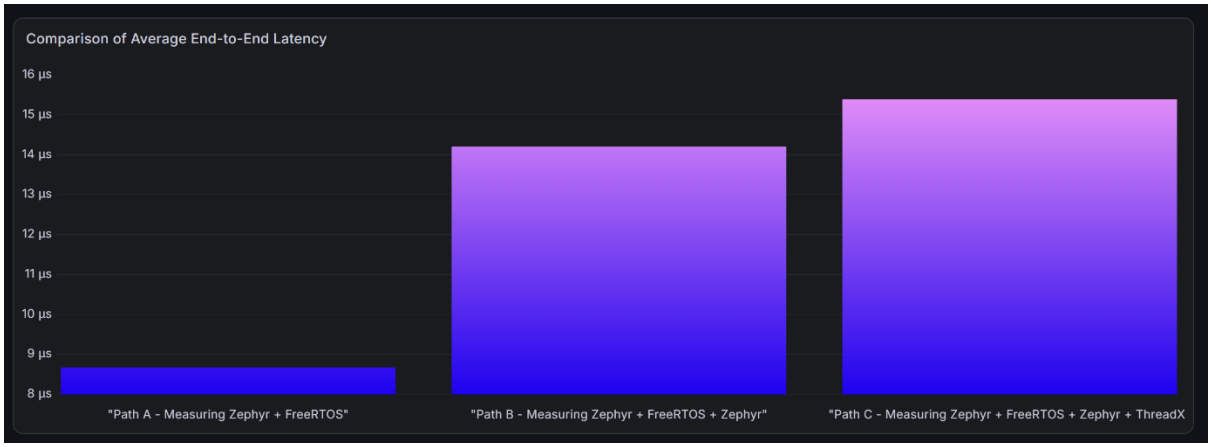

Figure 5.7 – Comparison of average end-to-end latency across communication paths

Figure 5.7 compares the average end-to-end latency values obtained from the three evaluated communication paths. Path A achieved an average latency of 1456 cycles, corresponding to approximately 8.66 µs. Path B increased to 2384 cycles, corresponding to approximately 14.19 µs. Path C recorded the highest average latency of 2585 cycles, corresponding to approximately 15.38 µs.

Since the evaluated architecture follows a daisy-chain communication structure, these results should not be interpreted as a direct ranking of individual RTOS performance. Instead, the increasing latency reflects the cumulative overhead introduced by additional forwarding nodes and RTOS boundaries along the communication path.

5.2.6 Comparative Latency Distribution

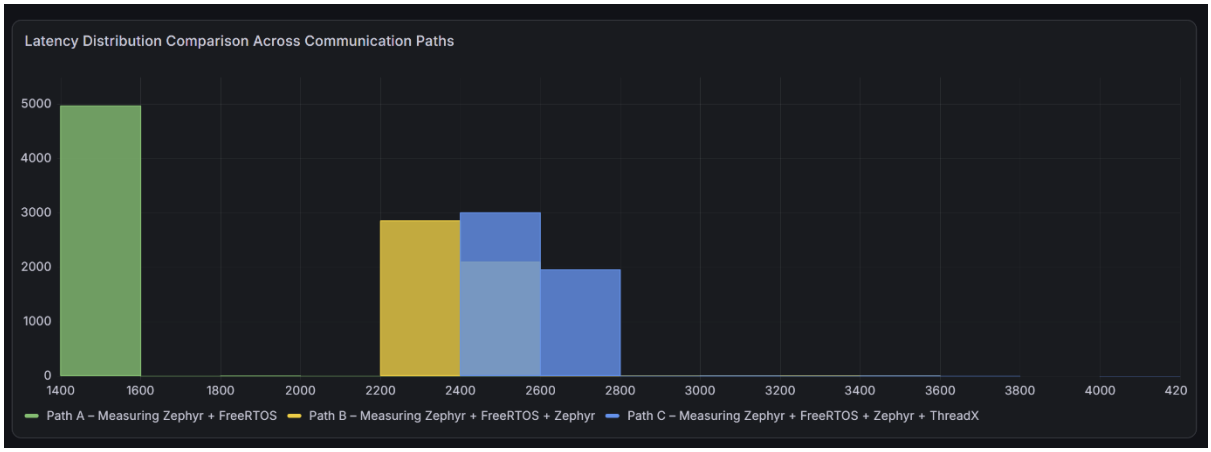

Figure 5.8 – Comparative latency distribution across communication paths.

Figure 5.8 compares the latency distributions of all evaluated communication paths. The distributions are clearly separated, with Path A concentrated around 1450 cycles, Path B around 2400 cycles, and Path C around 2600 cycles. This separation demonstrates the incremental latency introduced as the signal propagates through additional forwarding stages.

The histograms also show that most measurements are tightly concentrated around their dominant latency regions. A small number of outliers are visible in each path, indicating occasional timing jitter; however, these outliers represent only a small portion of the total 5000 samples.

5.2.7 Oscilloscope-Based Validation

In addition to the software-based latency measurements obtained using the STM32 hardware cycle counter, an independent hardware-level validation was performed using a digital oscilloscope. The objective was to verify that the latency values reported by the benchmark framework correspond to the actual signal propagation delay observed on the communication path.

For this validation, the complete communication chain of Path C was evaluated. Two GPIO signals were monitored simultaneously: the trigger signal generated by the measurement node and the return signal received after propagation through the forwarding nodes. Cursor measurements were used to determine the time difference between the two signal transitions.

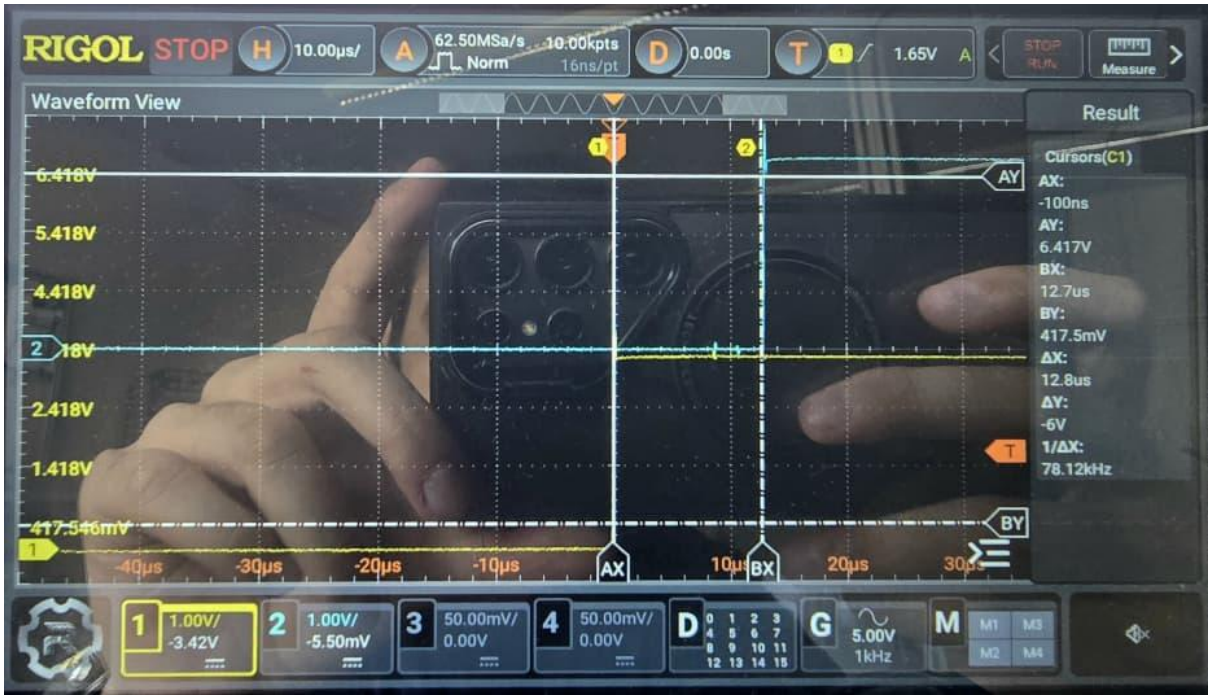

Figure 5.9 – Oscilloscope measurement of end-to-end latency for Path C

difference of approximately 12.8 µs (Δ x) between the transmitted and received signals. This value is in the same order of magnitude as the average latency obtained from the cycle counter measurements for Path C (15.38 µs).

The difference between the two measurements can be attributed to several factors, including oscilloscope cursor placement accuracy, GPIO signal transition timing, interrupt processing overhead, and software execution delays that are captured by the cycle-counter benchmark but may not be fully represented by the selected signal edges observed on the oscilloscope.

Nevertheless, the oscilloscope results confirm the validity of the software-based measurement methodology and demonstrate that the reported latency values accurately reflect the timing behaviour of the distributed RTOS communication system.

5.3 Discussion

The experimental results demonstrate the impact of increasing communication complexity on end-to-end latency within the distributed RTOS architecture. As additional forwarding nodes were introduced into the communication chain, the measured latency increased accordingly. This behaviour is expected because each forwarding node contributes additional interrupt handling, signal processing, GPIO operations, and software execution overhead.

Path A, which involved communication between the measurement node and a single FreeRTOS forwarding node, achieved the lowest average latency of 1456 cycles (approximately 8.66 µs). The corresponding time-series plot showed a highly stable latency profile, with most measurements concentrated near the average value and only

a limited number of latency spikes. The histogram distribution further confirmed that the majority of samples were tightly clustered within a narrow latency range, indicating predictable timing behaviour.

When the communication path was extended in Path B by adding a Zephyr forwarding node, the average latency increased to 2384 cycles (approximately 14.19 μs). This increase reflects the additional processing stages introduced into the communication chain. Despite the increased latency, the measured results remained stable, and the distribution of latency values remained concentrated around the average value. The limited number of higher-latency samples suggests that the system maintained a high degree of determinism under the evaluated workload.

Path C represented the most complex communication scenario, incorporating FreeRTOS, Zephyr, and ThreadX forwarding nodes within the same communication path. As expected, this configuration produced the highest average latency of 2585 cycles (approximately 15.38 μs). Although the latency increased compared with the other configurations, the increase was relatively modest considering the additional forwarding stage. The time-series and histogram results indicated that the majority of measurements remained tightly grouped around the mean value, demonstrating that the distributed architecture preserved predictable timing characteristics even when multiple RTOS platforms were integrated into the communication chain.

An important observation is that the increase in average latency was not proportional to the number of forwarding nodes. The transition from Path A to Path B introduced a substantial increase in latency, whereas the additional ThreadX node in Path C contributed a comparatively smaller increase. This suggests that the communication overhead is influenced not only by the number of forwarding stages but also by the implementation details of interrupt handling, GPIO response time, and RTOS scheduling behaviour within each node.

The histogram distributions revealed that most latency samples were concentrated within narrow regions around their respective mean values. While occasional latency spikes were observed in all communication paths, these events represented only a small fraction of the total 5,000 collected samples. Consequently, the overall timing behaviour can be considered deterministic and suitable for real-time distributed communication applications.

The oscilloscope-based validation provided additional confidence in the measurement methodology. Independent hardware measurements produced latency values of the same order of magnitude as those obtained from the STM32 hardware cycle counter. Although small differences were observed between the two measurement techniques, the results confirmed the validity of the software-based benchmarking approach and demonstrated that the measured latency values accurately represent the behaviour of the physical communication system.

Overall, the results indicate that all evaluated RTOS configurations were capable of supporting low-latency communication within the proposed distributed architecture. While latency increased as additional forwarding nodes were introduced, the measured

delays remained within the microsecond range and exhibited stable and predictable timing behaviour. These findings demonstrate the feasibility of integrating FreeRTOS, Zephyr, and ThreadX within a heterogeneous distributed embedded system while maintaining deterministic communication performance

5.4 Chapter Summary

This chapter presented the experimental evaluation of the proposed distributed RTOS communication architecture. Three communication paths were analysed using FreeRTOS, Zephyr, and ThreadX forwarding nodes under identical hardware conditions. For each configuration, 5,000 latency measurements were collected using the STM32 hardware cycle counter and subsequently analysed using statistical and graphical methods.

The results showed that Path A achieved the lowest average end-to-end latency of 1456 cycles (8.66 µs), while Path B and Path C produced average latencies of 2384 cycles (14.19 µs) and 2585 cycles (15.38 µs), respectively. As additional forwarding nodes were introduced into the communication chain, the overall communication latency increased, reflecting the additional processing and forwarding overhead associated with each RTOS node.

Time-series plots and histogram distributions demonstrated that latency values remained concentrated around their respective average values throughout all benchmark executions. Although occasional latency spikes were observed, the majority of measurements exhibited stable and predictable timing behaviour. The limited variability observed across the collected datasets indicates that the proposed architecture maintains deterministic communication characteristics suitable for real-time distributed embedded applications.

The oscilloscope-based validation further confirmed the accuracy of the software-based measurements obtained from the hardware cycle counter. The independently measured propagation delays were consistent with the calculated latency values, providing additional confidence in the validity of the benchmarking methodology.

Overall, the evaluation results demonstrate that heterogeneous RTOS platforms can be successfully integrated within a distributed embedded communication architecture while maintaining low-latency and deterministic operation. The findings provide a quantitative basis for the comparative analysis presented in the following chapter.

Chapter 6 – Conclusion and Future Work

6.1 Conclusion

This research investigated the latency characteristics of multiple Real-Time Operating Systems (RTOSs) operating within a distributed embedded communication architecture. A benchmark platform consisting of four STM32 Nucleo-F439ZI development boards was implemented using FreeRTOS, Zephyr RTOS, and Azure RTOS ThreadX. Unlike many previous RTOS studies that evaluate operating systems in isolation, this work focused on end-to-end communication latency across a heterogeneous multi-node system.

To ensure accurate timing measurements, the ARM Cortex-M4 hardware cycle counter was used to record latency at cycle-level resolution. A total of 5,000 measurements were collected for each experimental scenario, and the resulting datasets were analysed using statistical methods and Grafana-based visualizations. In addition, oscilloscope measurements were used to validate the accuracy of the software-based timing methodology.

The experimental results demonstrated that latency increased as additional forwarding nodes were introduced into the communication chain. Path A, consisting of the measurement node and a FreeRTOS forwarding node, achieved the lowest average latency of 1456 cycles (approximately 8.66 μs). Path B, which added a Zephyr forwarding node, increased the average latency to 2384 cycles (approximately 14.19 μs). The complete communication chain evaluated in Path C produced an average latency of 2585 cycles (approximately 15.38 μs).

Although the overall latency increased with system complexity, all evaluated communication paths exhibited stable behaviour with relatively small timing variations. The absence of significant outliers and the consistency observed in the time-series and histogram analyses indicate that the proposed architecture maintained predictable communication performance throughout the experiments.

The results confirm that FreeRTOS, Zephyr, and ThreadX can successfully operate together within a distributed embedded environment while maintaining deterministic timing behaviour. Furthermore, the study demonstrates that hardware cycle-counter measurements combined with interrupt-driven communication provide an effective methodology for evaluating RTOS performance in practical multi-node systems.

Overall, the research contributes a practical framework for analysing end-to-end latency in heterogeneous RTOS-based embedded systems and provides useful insights for designers of industrial, IoT, and real-time distributed applications.

6.2 Future Work

Several opportunities exist for extending the work presented in this research.

First, future studies could investigate additional RTOS platforms, such as RTX5, embOS, RTEMS, or VxWorks, to provide a broader comparison of real-time performance characteristics.

Second, the current implementation relies on GPIO-based communication. Future work could evaluate more realistic communication mechanisms, including UART, SPI, I2C, CAN, Ethernet, or industrial communication protocols, to assess their impact on end-to-end latency.

Third, the benchmark could be extended to include larger communication chains consisting of additional nodes. This would allow investigation of latency scalability and the cumulative effects of distributed processing in more complex embedded architectures.

Fourth, future research could measure additional RTOS performance metrics, including interrupt latency, context-switching time, task scheduling overhead, semaphore performance, memory allocation latency, and inter-task communication delays.

Finally, the experiments were performed using STM32F439ZI microcontrollers operating at a fixed clock frequency. Future studies could evaluate different processor architectures and hardware platforms to determine how device-specific characteristics influence distributed RTOS communication performance.

References

[1] R. R. Belleza, A. Dadios, and J. M. Martinez, “Performance Study of Real-Time Operating Systems for Internet of Things Devices,” IET Wireless Sensor Systems, vol. 8, no. 6, pp. 261–268, Dec. 2018.

[2] M. Mazzi, M. Piumatti, and A. Macii, “Benchmarking and Comparison of Two Open-Source RTOSs for Embedded Systems Based on ARM Cortex-M4 MCU,” Indian Journal of Science and Technology, vol. 14, no. 17, pp. 1403–1414, 2021.

[3] S. Huber, “Measuring Real-Time Operating System Performance – Part II: Comparing FreeRTOS vs. Zephyr,” UL Solutions Software Integrity Group Blog, Sep. 2021. [Online]. Available: https://www.ul.com/sis/blog/measuring-real-time-operating-system-performance-part-ii-comparing-freertos-vs-zephyr [Accessed: 22-Jun-2026].

[4] J. Beningo, "RTOS Performance Report (2024)," Beningo Embedded Group, Sep. 2024. [Online]. Available: https://www.beningo.com/rtos-performance-report-2024-now-available/. [Accessed: Jun. 22, 2026].

[5] J. Beningo, "How Do You Test RTOS Performance?" Embedded Computing Design, Oct. 2024. [Online]. Available: https://www.embedded.com/how-do-you-test-rtos-performance/. [Accessed: Jun. 22, 2026].

[6] Promwad, “Choosing an RTOS for Your Device: Comparing FreeRTOS, Zephyr, ThreadX, and More,” Apr. 17, 2025. [Online]. Available: https://promwad.com/news/choosing-rtos-freertos-zephyr-threadx-comparison [Accessed: Jun. 22, 2026]

[7] P. H. Truong, H. Q. T. Ngo, and V. D. T. Tran, “Review of Azure RTOS ThreadX Real-Time Operating System on STM32 Platforms,” EAI Endorsed Transactions on Sustainable Manufacturing and Renewable Energy, 2025. [Online]. Available: https://publications.eai.eu/index.php/sumare/article/view/11710 [Accessed: Jun. 22, 2026].